当代码不再稀缺：AI 放大效应与确定性工程

标准录音 14录音 • 2026-10-23 16:48 - 17:34

一、AI 放大效应与大型工程挑战

AI 放大器的双刃剑效应

对于 Solo 开发者或研究场景，AI 是能力增强工具，可快速完成 POC 验证；但在大型工程中，**AI 是短板的放大器**——以前做得好与不好的地方都会被 100 倍放大。Code Agent 会复制现有框架和架构，好的坏的一并放大，如同 **500 个资深程序员同时加入团队**，但文档一致性、CI 流水线吞吐能力、代码审查资源均无法匹配。

规则破坏的放大效应

规则写在文档中靠人记忆，AI 放大后规则破坏性同样放大。以 iOS 和安卓团队为例，即使需求几乎一致，边界交流问题在 10 倍、20 倍放大后更加突出。**Code Agent 不会帮助补全规则，只会原样复制现有模式**。

大型工程规模数据（B 站）

- 以 B 站最大仓库为例，APP 编译总数为 **14.3 万个 Action**（如 Gradle 依赖图节点、编译单元）；
- Action 依赖深度达 **64 层**，需串行处理；
- 人员规模方面，九成以上（保守估计）员工使用该仓库。

二、确定性工程四大支柱

核心框架：**约束（Constraint）→ 定位（Location）→ 验证（Verification）→ 度量（Measurement）**，必须严格按 1-2-3-4 顺序执行。

1. 约束（Constraint）

- 通过**强类型 IDL**（如 Protobuf）统一接口定义：后端 RPC 服务统一使用 B 站自研 BRPC 框架；前端通过 IDL 自动生成各平台代码；**编译失败即阻断上线**；
- 所有 RPC、跨语言调用、跨进程通信必须用强类型 IDL 描述，禁止弱类型传递（如 Kafka 消息、JNI/NAPI 裸写），从源头杜绝错误；
- 语言层面约束：禁止回调式异步（Callback），统一使用协程（如 Kotlin Coroutine），流程从链条式转为同步式，资源生命周期由语言保证；
- 本地与 CI 环境完全一致：通过 Bazel 构建实现**全工具链密封**，本地构建与 CI 流程完全一致，确保 AI 在本地可发现所有问题。

2. 定位（Location）

基于 Meta Query 能力实现精准定位：

- QA 可查询测试范围；
- APP 可查询模块依赖关系；
- 底层模块改动可自动识别受影响模块和所需测试数量；
- 通过 Action Query 可查询 14.3 万个 Action 中每个编译单元的具体编译参数，AI 可快速复现编译问题。

3. 验证（Verification）

- **Web 端**：通过 Playwright 打开页面，结合 DOM 结构验证功能是否实现；
- **后端**：通过接口返回数据验证；
- **客户端**：因编译时间过长（数十分钟），验证成本高。B 站通过将安卓 Gradle 迁移至 Bazel、iOS 构建改为分布式构建（Remote Worker），**日构建次数翻倍**。

4. 度量（Measurement）

去除所有表面指标（如 Token 使用量），聚焦实际业务价值，**不设虚荣指标**。

三、客户端验证困境

实验数据

在某版本中拉取 50 个最适合 AI 写的迭代需求，最终**仅 5 个被合并**，Human-out-of-the-loop 模式失败。原因在于：所有合并需求均存在兜底和后门机制，无法 AB Test 的需求（即使仅一行代码）必须人工确认。

技术瓶颈

安卓可通过 Linux 虚拟化构建虚拟 Worker，但 **iOS Simulator 无并发能力**，**鸿蒙无虚拟化方案**，导致客户端端到端验证缺乏基础设施——这是当前最大的技术难点。

四、组织架构与业务背景

B 站完成 **Full Stack 化**组织架构调整：前端（Web + 媒体侧）与后端面向同一业务交付。架构分为三层：

1. **国内/出海业务层**：前端同学覆盖；
2. **中间层**：后端同学覆盖，处理大流量高并发；
3. **底层基础架构组件层**。

五、行业对比与最佳实践

Chrome 工程实践

Chrome 持续重构但遵守简单规范：**依赖方向明确**（上不依赖下）、**接口归属清晰**、**语义话语权明确**（如 AIDL、gRPC 的 IDL 描述）——通过简单规则实现长期健康发展。

行业工具对比

对比项	结论
Bazel vs Go Module	Bazel 依赖分析能力强 10 倍
本地构建与 CI 一致性	Bazel 实现全公司共享构建缓存

六、AI 应用实践反思

- **内部 AI 工具建设**：尝试建立低价规范模型（国产模型），效果远不如第三方；尝试需求侧改造（从上游压力开始），结论是**应先管好技术再管产品**，已放弃该方向；
- **未来规划**：持续建设 Verify 能力（客户端验证）；后端因大流量业务**不敢将发布权交给 Code Agent**，保持人工审核。

七、重点与难点

重点

- **确定性工程四大支柱**：约束、定位、验证、度量，是解决 AI 放大效应的核心框架，必须严格按顺序执行（1-2-3-4）；
- **强类型 IDL 约束**：所有 RPC、跨语言调用、跨进程通信必须使用强类型 IDL 描述，禁止弱类型传递（如 Kafka 消息、JNI/NAPI 裸写），编译失败即阻断，从源头杜绝错误；
- **本地与 CI 环境一致性**：通过 Bazel 实现全工具链密封，本地构建与 CI 流程完全一致，确保 AI 在本地可发现所有问题。

难点

- **客户端验证瓶颈**：iOS Simulator 无并发能力、鸿蒙无虚拟化方案，导致客户端端到端验证缺乏基础设施，是当前最大技术难点；
- **Human-out-of-the-loop 失败**：即使端到端验证全部通过，开发者仍不敢合并代码，需人工确认，说明信任机制尚未建立；
- **AI 幻觉与文档腐化**：文档更新不及时导致 AI 学习错误信息，老规则（如 Prompt、Agent 配置）在模型更新后可能失效，需持续维护。

由 NEXT AI 生成，内容仅供参考